

PORTABLE ECG/EKG MONITOR SEDERHANA
MATA KULIAH: MIKROPROSESOR DAN ANTARMUKA



DOSEN PEMBIMBING:

Agus Virgono, Ir., MT.

DISUSUN OLEH

Nazmi Hilmi Nuruzzaman (101032300164)

Rofly Hi M Naser (101032300085)

FAKULTAS TEKNIK ELEKTRO

TELKOM UNIVERSITY

2025/2026

I. Latar Belakang

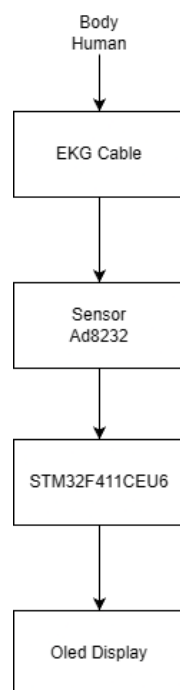
Mikrokontroler STM32 dengan arsitektur ARM Cortex-M4 dipilih untuk proyek ECG monitoring karena memiliki kecepatan proses yang memadai untuk mengolah sinyal jantung, konsumsi daya yang efisien untuk perangkat portable, dan peripheral lengkap seperti ADC, timer, dan interface I2C yang terintegrasi. Hal ini memungkinkan sistem ECG yang sederhana namun fungsional tanpa perlu tambahan komponen eksternal yang banyak.

Elektrokardiogram (ECG) bekerja dengan mendeteksi aktivitas listrik jantung melalui elektrode yang ditempelkan pada kulit. Sensor ini mampu menangkap pola gelombang jantung khas seperti kompleks QRS yang menandakan detak, serta gelombang P dan T yang memberikan informasi tentang ritme dan kesehatan jantung secara umum.

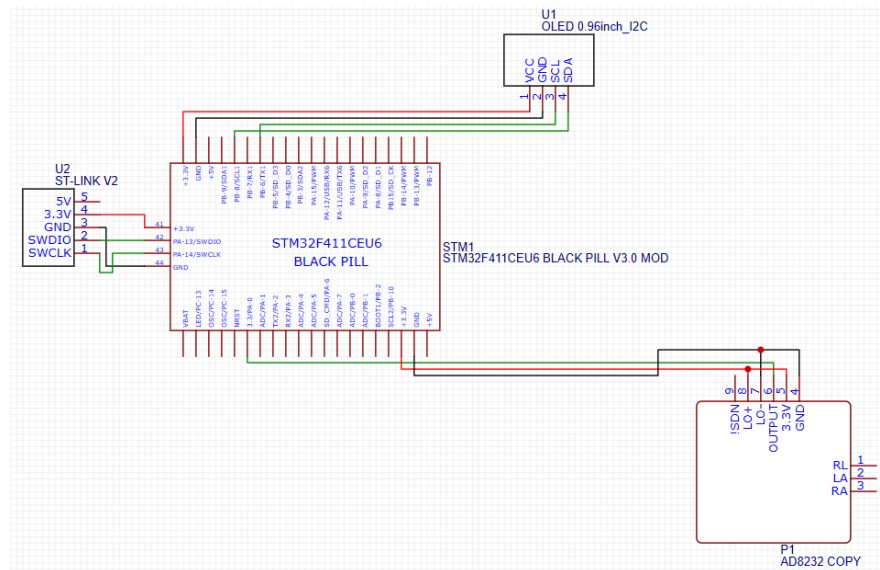
Dalam implementasinya, sistem terdiri dari tiga bagian utama: sensor ECG sebagai input, STM32 sebagai processor, dan OLED display sebagai output. Komunikasi antar modul menggunakan I2C yang hemat pin GPIO. Pemrograman dilakukan dengan pendekatan level register untuk optimasi performa dan memahami arsitektur mikrokontroler secara mendalam, meskipun lebih kompleks dibanding penggunaan library standar.

Proyek ini relevan dengan tren wearable health devices yang sedang berkembang, sekaligus memberikan pengalaman praktis dalam embedded system untuk aplikasi kesehatan dengan biaya terjangkau dan kompleksitas yang sesuai untuk level mahasiswa teknik elektro atau biomedical.

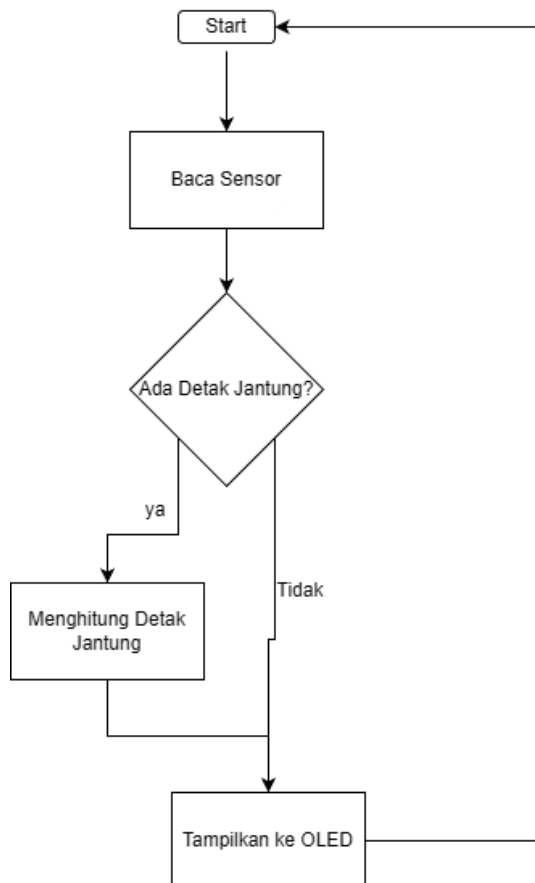
II. Diagram Blok Sistem



III. Skematik Rangkaian



IV. Flowchart



V. Source Code

```

/* USER CODE BEGIN Header */
/**

```

```

*****
*****
* @file      : main.c
* @brief     : Main program body

*****
*****
* @attention
*
* Copyright (c) 2026 STMicroelectronics.
* All rights reserved.
*
* This software is licensed under terms that can be found in the LICENSE file
* in the root directory of this software component.
* If no LICENSE file comes with this software, it is provided AS-IS.
*

*****
*****
*/
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include "ssd1306.h"
#include "ssd1306_fonts.h"
#include <stdio.h>
/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/

```

```

/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/
ADC_HandleTypeDef hadc1;

I2C_HandleTypeDef hi2c1;

/* USER CODE BEGIN PV */
uint32_t adc_value;
int x_pos = 0;
int y_old = 0;

uint32_t last_beat_time = 0;
uint32_t current_time = 0;
float bpm = 0;
int threshold = 3000;    // Sesuaikan nilai ini berdasarkan sinyal Anda
int is_beat = 0;        // Flag untuk mencegah double counting

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_ADC1_Init(void);
static void MX_I2C1_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{

```

```

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */

/* MCU Configuration-----*/

/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_ADC1_Init();
MX_I2C1_Init();
/* USER CODE BEGIN 2 */
ssd1306_Init();
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    // 1. Baca Nilai ADC dari Sensor AD8232
    HAL_ADC_Start(&hadc1);
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK) {
        adc_value = HAL_ADC_GetValue(&hadc1);
    }
    HAL_ADC_Stop(&hadc1);

    current_time = HAL_GetTick(); // Ambil waktu sekarang dalam milidetik

    // 2. Deteksi Puncak (Peak Detection)

```

```

    if (adc_value > threshold && is_beat == 0) {
        uint32_t duration = current_time - last_beat_time;

        // Filter: Manusia normal BPM 40-200 (durasi 300ms - 1500ms)
        if (duration > 300) {
            bpm = 60000.0 / duration; // Hitung BPM
            last_beat_time = current_time;
            is_beat = 1; // Tandai bahwa puncak sudah terdeteksi
        }
    }

    // Reset flag saat sinyal turun di bawah threshold
    if (adc_value < (threshold - 200)) {
        is_beat = 0;
    }

    // 3. Tampilkan ke OLED setiap 200ms agar tidak berkedip (flicker)
    static uint32_t last_display_time = 0;
    if (current_time - last_display_time > 200) {
        char msg[15];
        sprintf(msg, "BPM: %.1f ", bpm); // Format angka ke string

        ssd1306_Fill(Black);
        ssd1306_SetCursor(10, 10);
        ssd1306_WriteString("HEART RATE", Font_7x10, White);

        ssd1306_SetCursor(10, 30);
        ssd1306_WriteString(msg, Font_11x18, White);

        ssd1306_UpdateScreen();
        last_display_time = current_time;
    }

    HAL_Delay(10); // Sampling rate 100Hz

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**

```

```

* @brief System Clock Configuration
* @retval None
*/
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();

    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    /** Initializes the RCC Oscillators according to the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue =
RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
        |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) !=
HAL_OK)
    {
        Error_Handler();
    }
}

```



```

}

/**
 * @brief ADC1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_ADC1_Init(void)
{

    /* USER CODE BEGIN ADC1_Init 0 */

    /* USER CODE END ADC1_Init 0 */

    ADC_ChannelConfTypeDef sConfig = {0};

    /* USER CODE BEGIN ADC1_Init 1 */

    /* USER CODE END ADC1_Init 1 */

    /** Configure the global features of the ADC (Clock, Resolution, Data
    Alignment and number of conversion)
    */
    hadc1.Instance = ADC1;
    hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV2;
    hadc1.Init.Resolution = ADC_RESOLUTION_12B;
    hadc1.Init.ScanConvMode = DISABLE;
    hadc1.Init.ContinuousConvMode = DISABLE;
    hadc1.Init.DiscontinuousConvMode = DISABLE;
    hadc1.Init.ExternalTrigConvEdge =
    ADC_EXTERNALTRIGCONVEDGE_NONE;
    hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    hadc1.Init.NbrOfConversion = 1;
    hadc1.Init.DMAContinuousRequests = DISABLE;
    hadc1.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
    if (HAL_ADC_Init(&hadc1) != HAL_OK)
    {
        Error_Handler();
    }

    /** Configure for the selected ADC regular channel its corresponding rank in the
    sequencer and its sample time.

```

```

*/
sConfig.Channel = ADC_CHANNEL_0;
sConfig.Rank = 1;
sConfig.SamplingTime = ADC_SAMPLETIME_3CYCLES;
if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN ADC1_Init 2 */

/* USER CODE END ADC1_Init 2 */

}

/**
 * @brief I2C1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_I2C1_Init(void)
{
    /* USER CODE BEGIN I2C1_Init 0 */

    /* USER CODE END I2C1_Init 0 */

    /* USER CODE BEGIN I2C1_Init 1 */

    /* USER CODE END I2C1_Init 1 */
    hi2c1.Instance = I2C1;
    hi2c1.Init.ClockSpeed = 100000;
    hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
    hi2c1.Init.OwnAddress1 = 0;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
    hi2c1.Init.OwnAddress2 = 0;
    hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
    hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
    if (HAL_I2C_Init(&hi2c1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN I2C1_Init 2 */

```

```

/* USER CODE END I2C1_Init 2 */

}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pins : PA1 PA2 */
    GPIO_InitStruct.Pin = GPIO_PIN_1|GPIO_PIN_2;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /* USER CODE BEGIN MX_GPIO_Init_2 */

    /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */

```

```

/* User can add his own implementation to report the HAL error return state */
__disable_irq();
while (1)
{
}
/* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

VI. Capture Input dan Output

